



Hump Yard Model

This presentation is a part of AnyLogic Standard Training Program



© The AnyLogic Company | www.anylogic.com

Hump Yard Model

- Build a model of a hump yard. A hump yard is a classification yard where cars are taken to a hump from where they are driven to classification tracks by the force of gravity.
- The arriving trains contain cars of 5 different types that are sent to 5 departure tracks. Once 9 cars of the same type accumulate on a departure track, they are driven away as a newly formed train.

Hump Yard. Phase 1

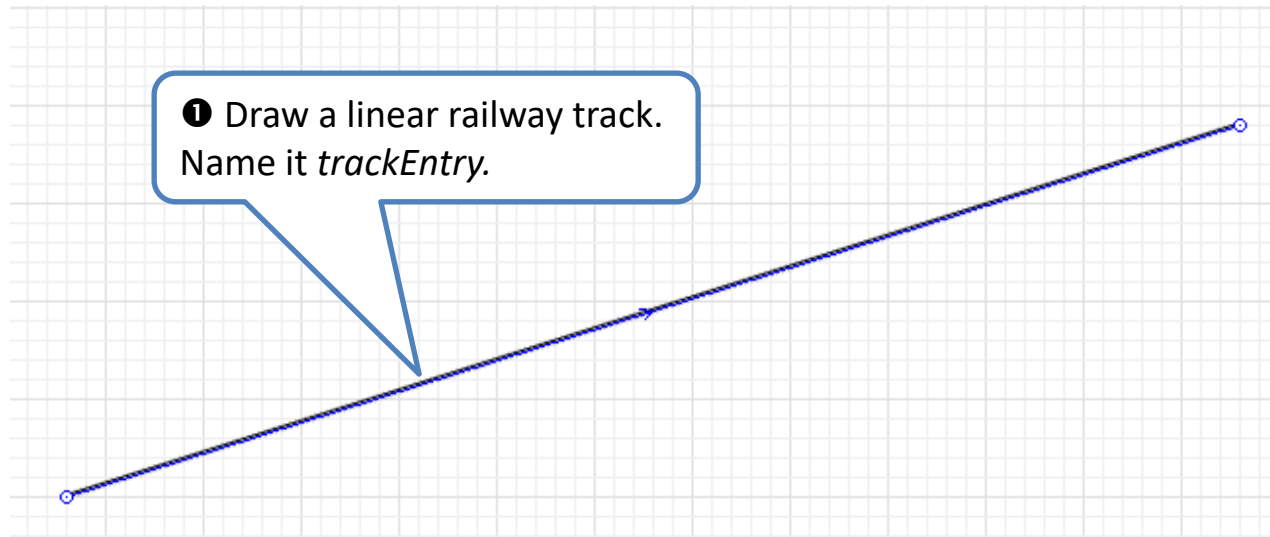
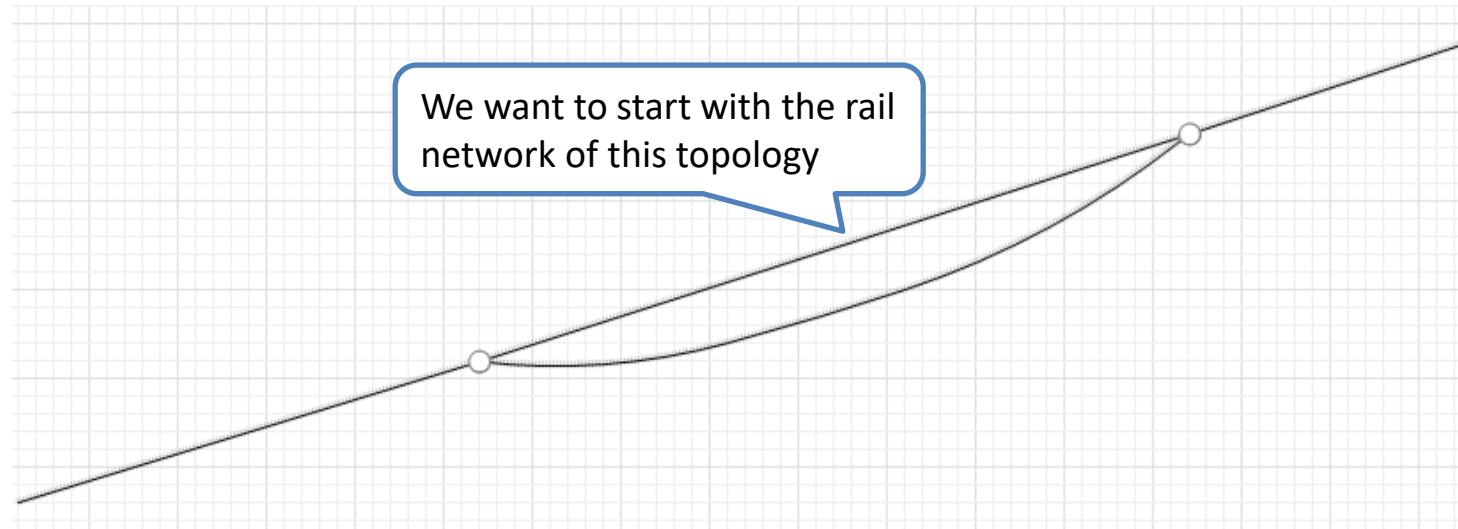
- Let's first simulate how the cars are taken to a track, sometimes called a **lead** or a **drill**. From there the cars are sent through a series of switches called a **ladder** onto the classification tracks.

We will build this model using AnyLogic Rail Library.

Rail Library

- **Rail Library** allows you to efficiently model, simulate and visualize operation of rail yards and rail transportation of any complexity and scale. Classification yards, rail yards of large plants, railway stations, rail car repair facilities, subways, airport shuttle trains, rail in container terminals, trams can be easily yet accurately modeled with this library.
- *Rail Library* is integrated well with *Process Modeling Library* and *Pedestrian Library*, which means you can combine rail models with models of trucks, cranes, ships, passenger flows, manufacturing or business processes, and so on.

Hump Yard. Phase 1. Step 1



Create a new model. Name it *Hump Yard*. Set the model time units to minutes.

Start defining the rail topology. First, draw the railway tracks.

Start with a linear track. The trains will appear on this track. That's why we name it *trackEntry*. We will give meaningful names only to the tracks that will be referred by the flowchart blocks. Other tracks may have their default names.

Drawing linear railway tracks

- You draw tracks in the same manner as paths. First, double-click the **Railway Track** element in the **Rail Library** palette.
- Then click in the graphical editor to start drawing the track.
- To draw a linear segment, just click where you want to place the segment end point.
- Finish drawing a track with a double-click.

Hump Yard. Phase 1. Step 2

❶ Draw a curved railway track starting from a point on the existing track. The circle denoting a switch will appear automatically.

❷ Finish drawing near the first track. You can not put the end point on the same track instantly.

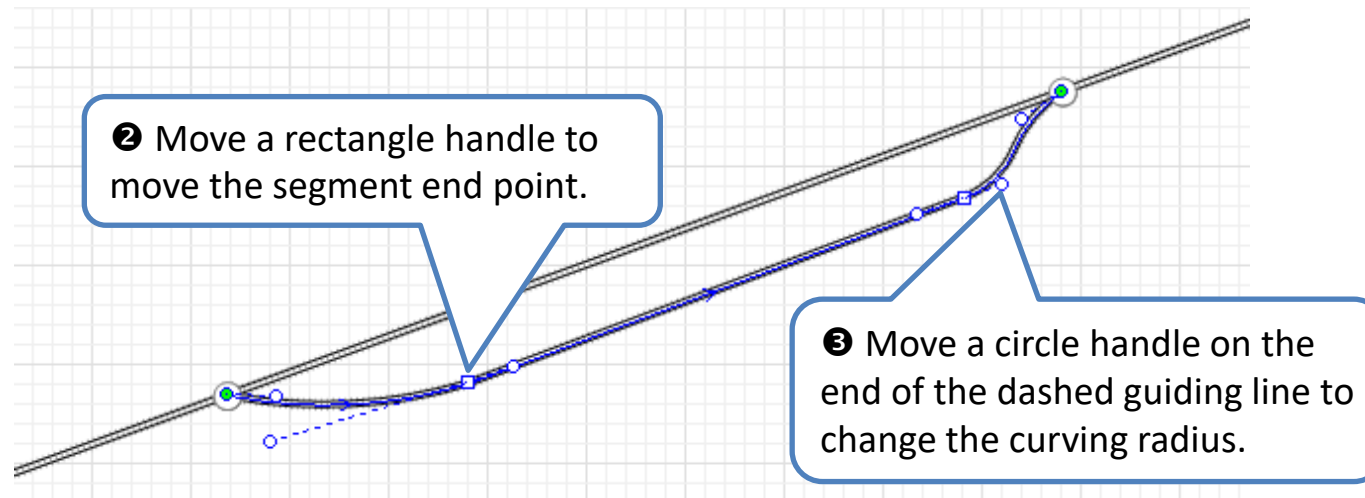
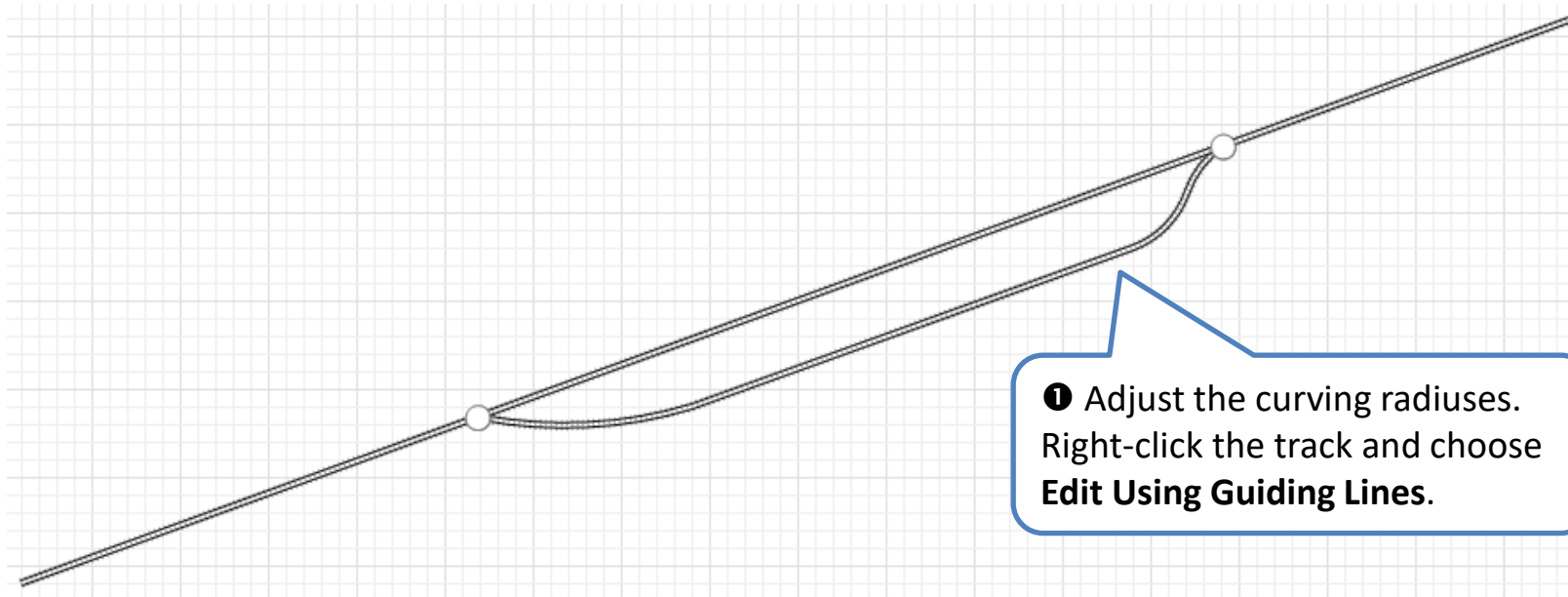
❸ When done, drag the track end point on the existing track to create a switch.

On the previous step we have drawn the linear track. However, tracks can be of curved shape. Let's now draw the curved lead track.

Drawing curved railway tracks

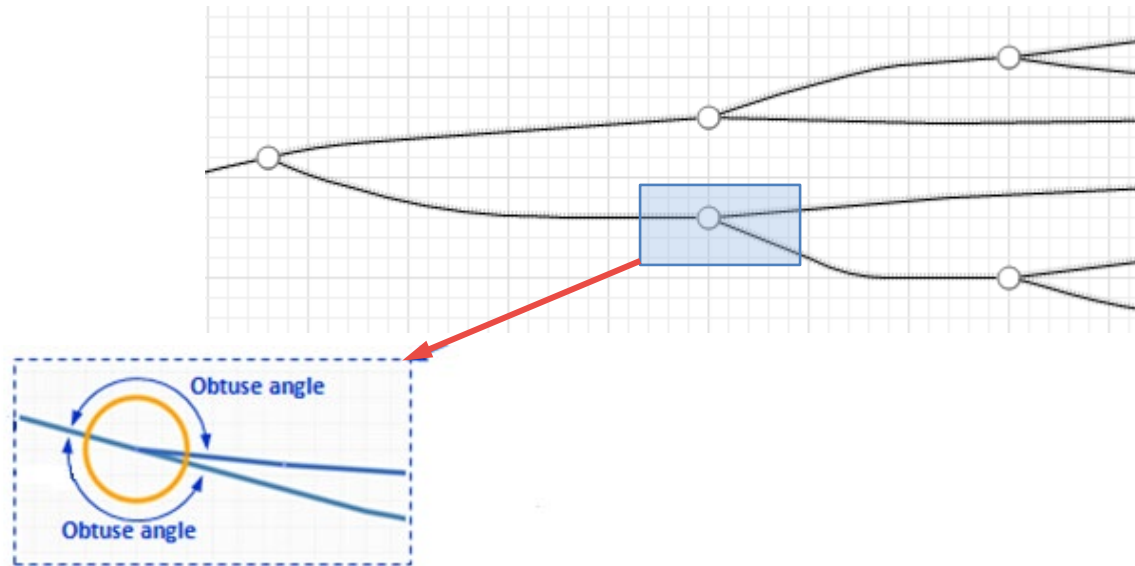
- Double-click the **Railway Track** element in the **Rail Library** palette. Then click in the graphical editor to start drawing the track.
- To append a curved segment, press the left mouse button at the point where the curved segment ends and then move the mouse with the button being pressed. While moving you will see how the curving radius changes. When done, release the mouse button.
- Finish drawing the track with a double-click.

Hump Yard. Phase 1. Step 3

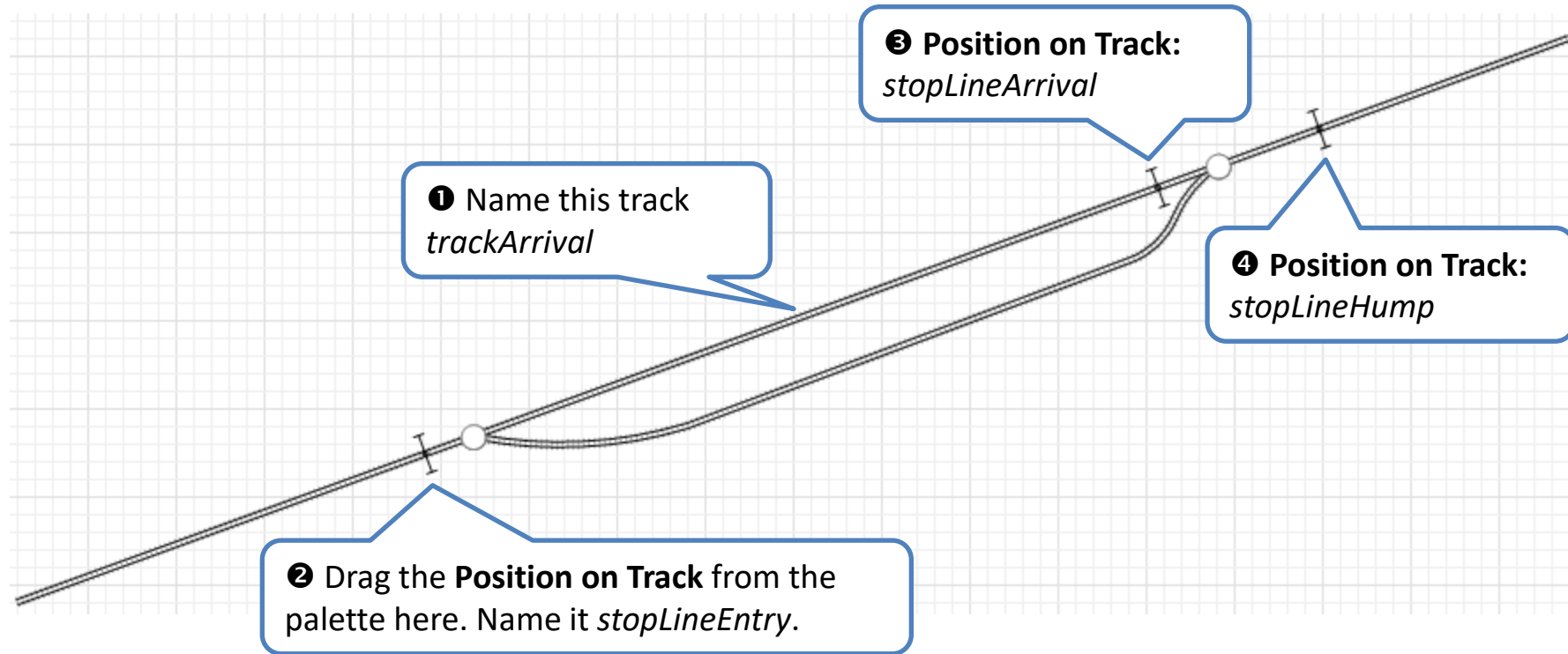


Railway switches

- Railway network consists of tracks and switches. While drawing rail networks, you will see that switches (circles) appear on the track end points.
- These are the rules explaining when a switch appears:
 - At each switch there must be at least two obtuse angles out of three between the track ends. The switch determines the routes based on those angles.



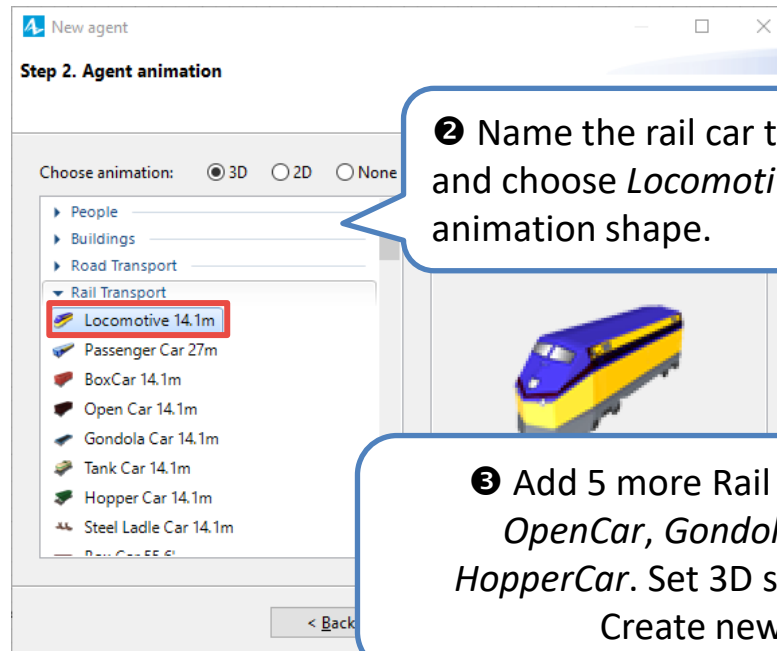
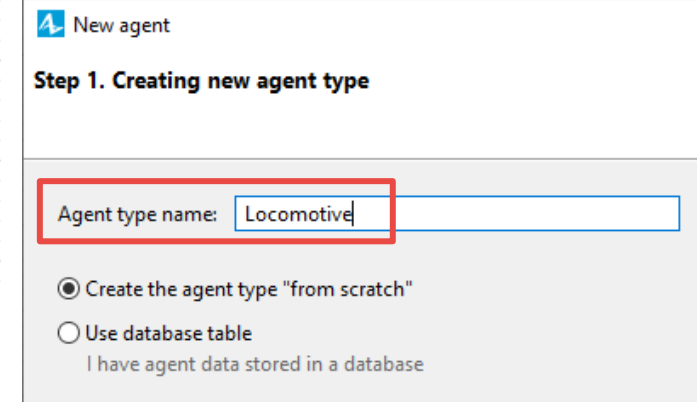
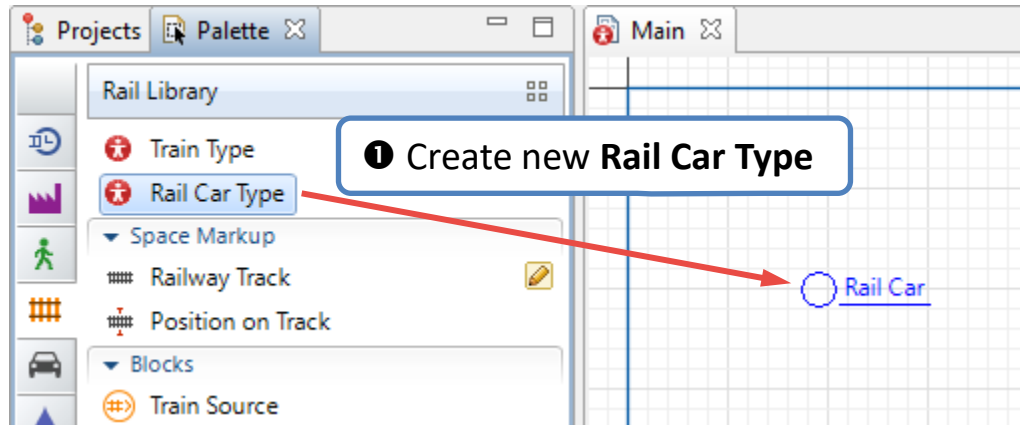
Hump Yard. Phase 1. Step 4



Position on track

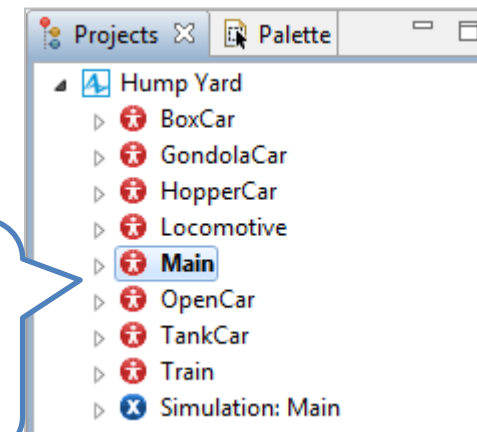
- **Position on Track** is a space markup shape that is used to define the exact position on the railway track. This is needed when you define:
 - *The position on the track where the train appears*
 - *The position where the train should stop*

Hump Yard. Phase 1. Step 5



2 Name the rail car type *Locomotive* and choose *Locomotive 14.1m* as its animation shape.

3 Add 5 more Rail Car types: *BoxCar*, *OpenCar*, *GondolaCar*, *TankCar* and *HopperCar*. Set 3D shapes for rail cars. Create new **Train Type: *Train*.**



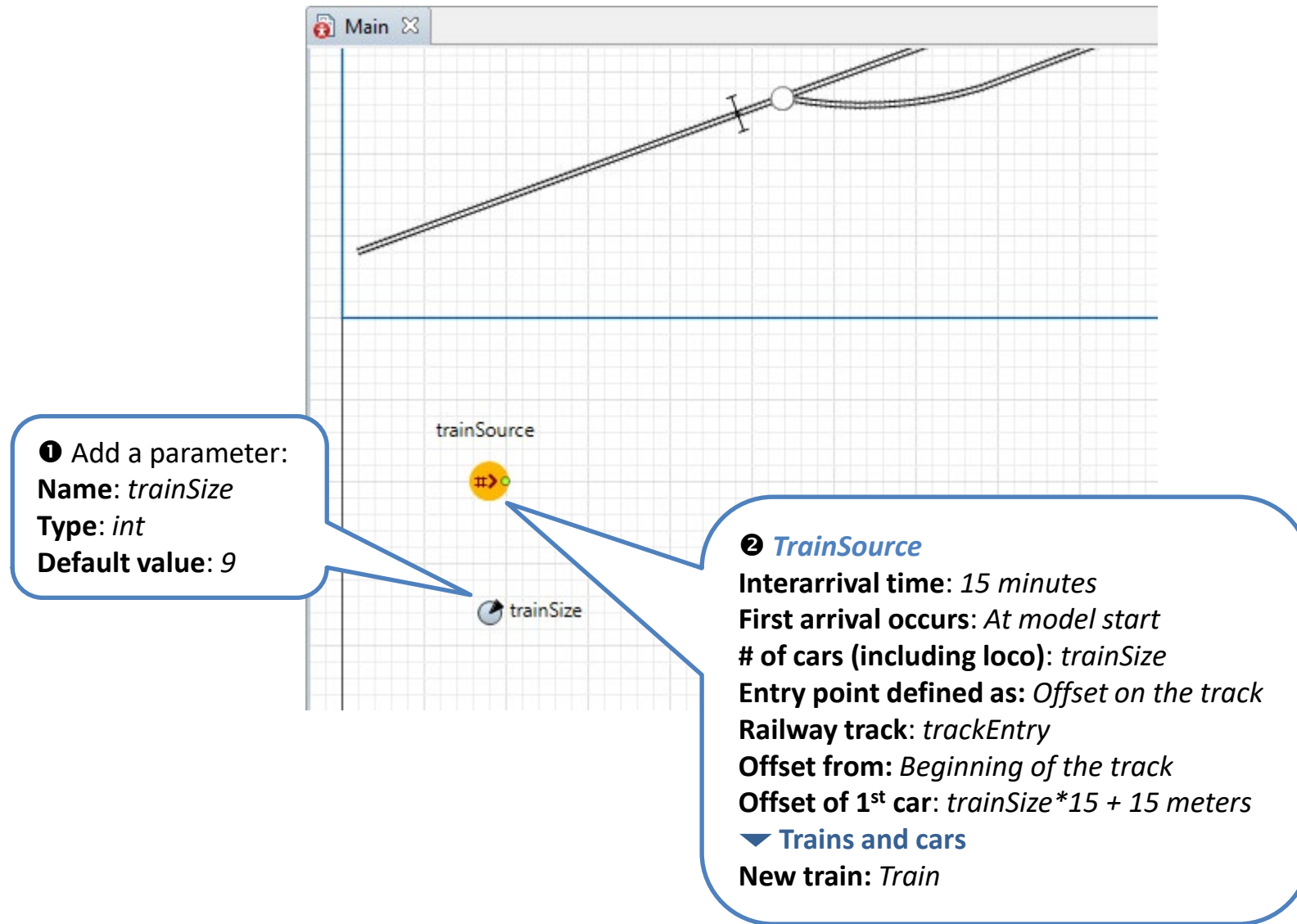
Create 6 rail car types: *Locomotive*, *BoxCar*, *OpenCar*, *GondolaCar*, *TankCar* and *HopperCar*. When creating new rail car types, choose the corresponding 3D animation objects from the list.

Similarly, create Train type (by dragging **Train Type** from the palette). Name it *Train*. You do not need to choose any animation figure for the train: it will consist of the cars.

Creating custom rail car types

- We provide you with the handy Rail Car Type Creation Wizard. Simply specify the name of the new rail car type and choose the animation from the list of ready-to-use 3D objects.

Hump Yard. Phase 1. Step 6



Let's start defining the logic using AnyLogic *Rail Library* blocks.

- ❶ Add an integer parameter defining the number of rail cars the train consists of.
- ❷ Start the flowchart with the train generator *TrainSource*. Trains will appear at track *trackEntry*.

Rail Library block	Description
Train Source	Creates trains, performs initial setup and puts them in the railway network. Starts any rail process flowchart. Supports several types of arrivals scheduling.
Train Dispose	Removes trains from the model.
Train Move To	Controls movement of trains. Can calculate routes and set switch states as the train goes along the route. Supports acceleration and deceleration.
Train Couple	Couples two trains that "touch" each other into one train.
Train Decouple	Decouples cars from the incoming train and creates a new train from those cars.
Train Enter	Takes the train agent and places it on the railway track.
Train Exit	Removes the train from the railway network and passes the train agent on to the regular process flowchart.
Rail Settings	Defines the advanced settings for the railway network.

Hump Yard. Phase 1. Step 7

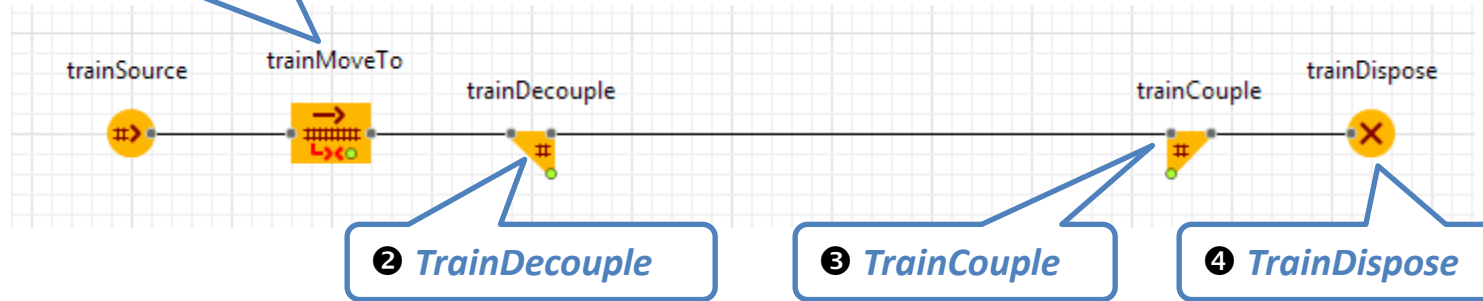
① *TrainMoveTo*

Route is: Calculated automatically from current to target track

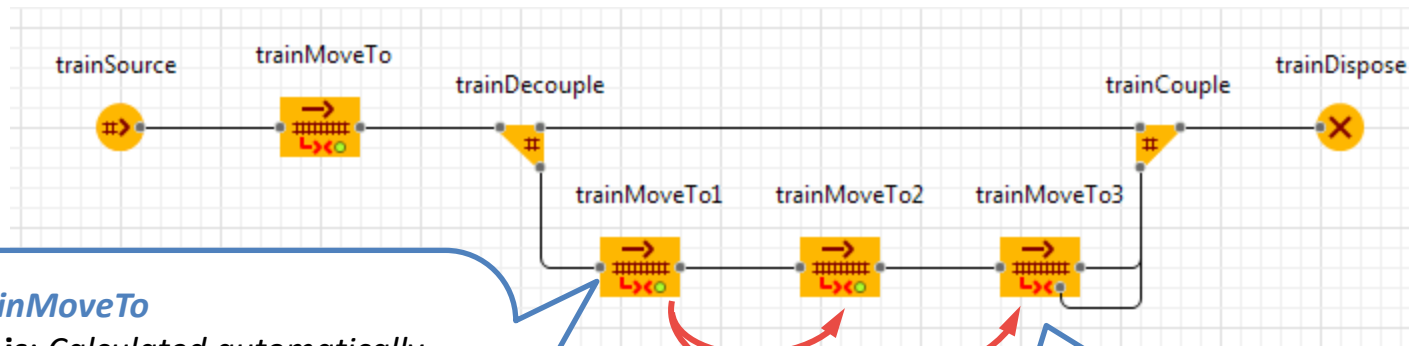
Target is: A given position on track

Position on track: *stopLineArrival*

Finish options: *Decelerate and stop*



Hump Yard. Phase 1. Step 8



① *TrainMoveTo*

Route is: Calculated automatically from current to target track

Target is: A given position on track

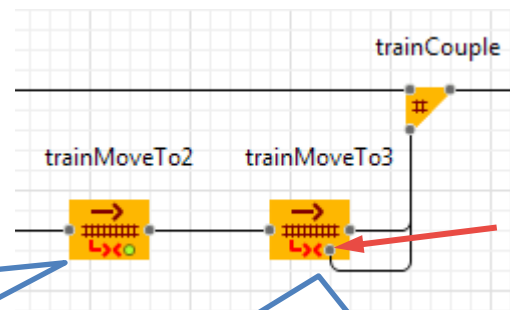
Position on track: *stopLineHump*

Cruise speed (0 for no change):

5 meters per second

Finish options: *Decelerate and stop*

② Create two more *TrainMoveTo* blocks by Ctrl+dragging (Mac OS: Cmd+dragging)



③ *Direction: Backward*

Route should not contain: *trackArrival*

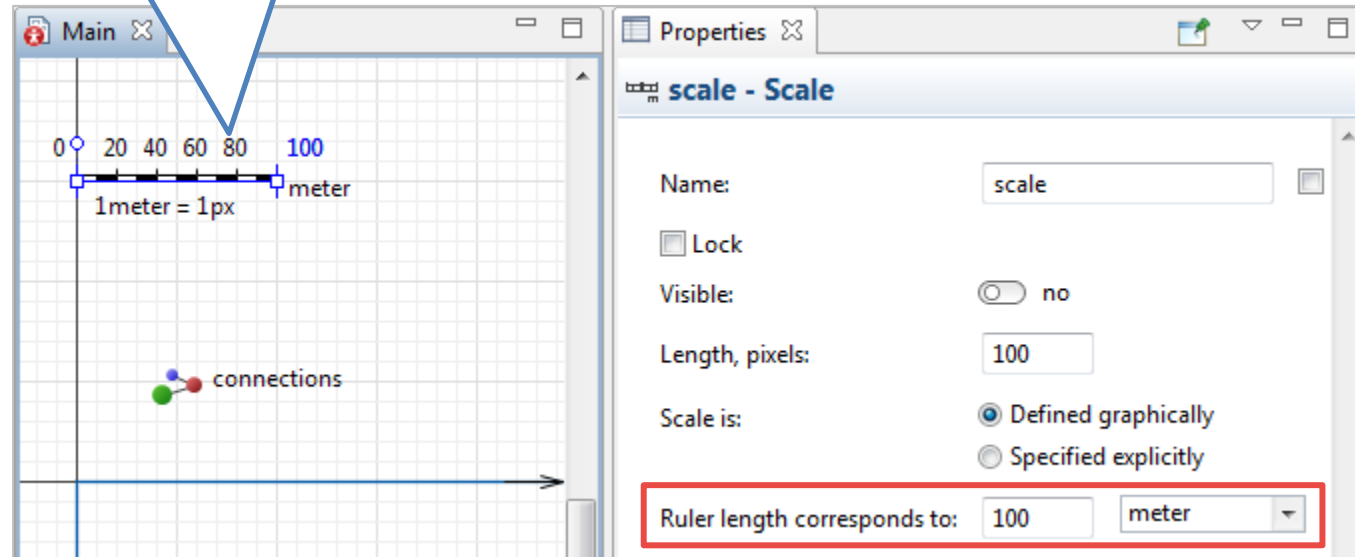
Position on track: *stopLineEntry*

④ **Position on track:**
stopLineArrival

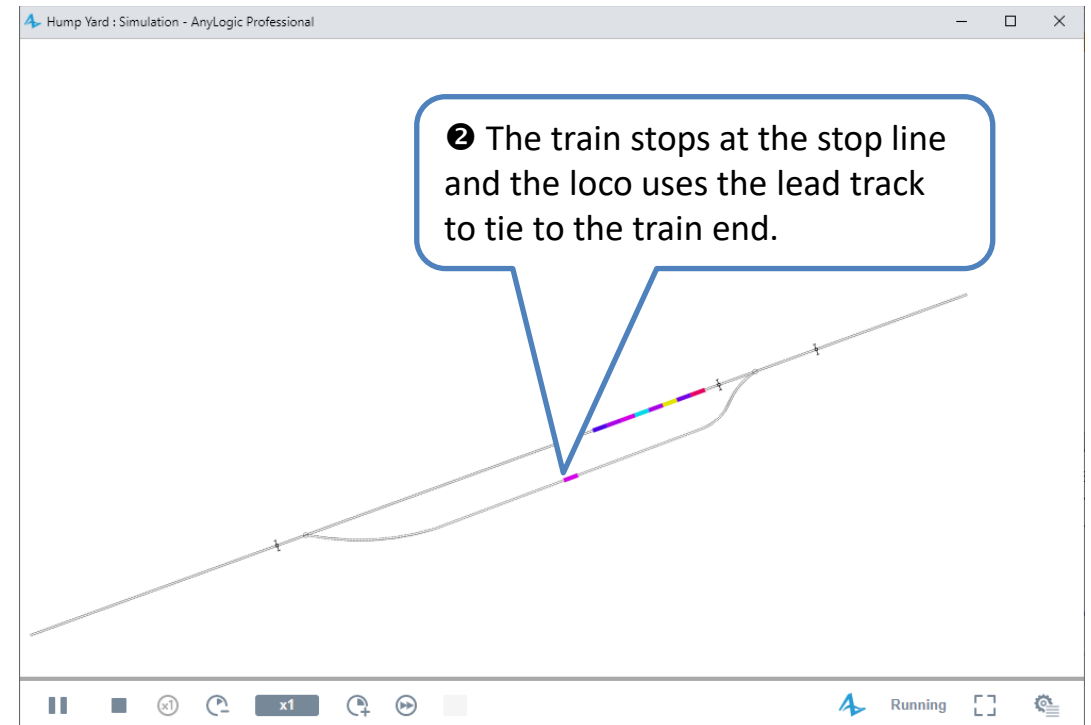
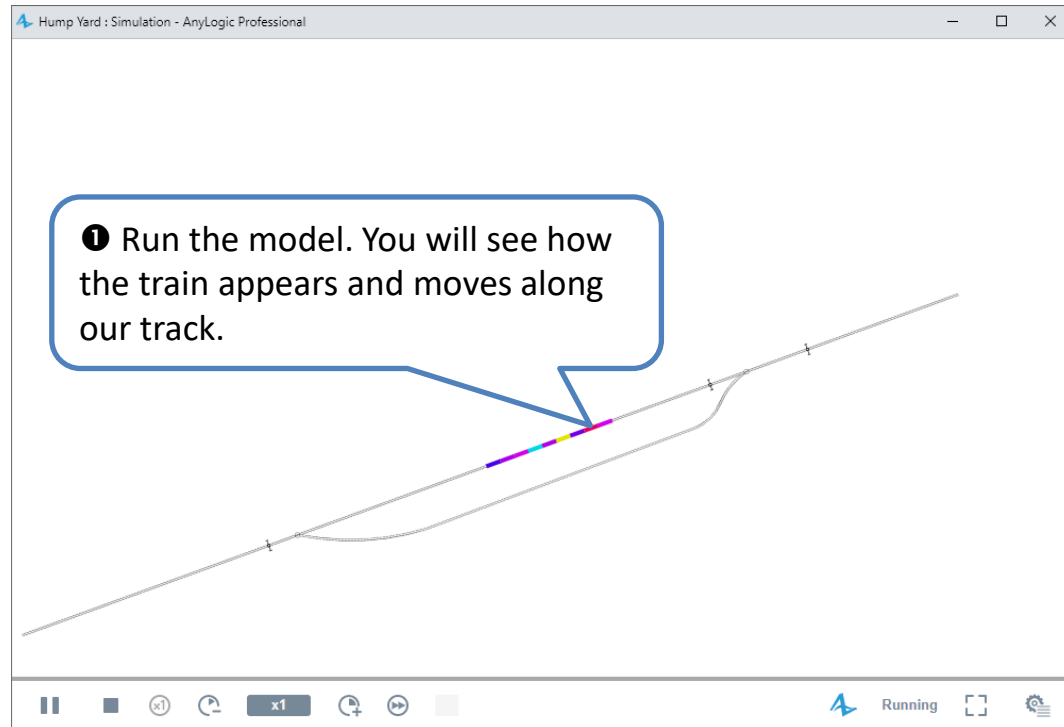
Hump Yard. Phase 1. Step 9

❶ Change the scale of the *Main* diagram. You can find the **Scale** element above the X-axis.

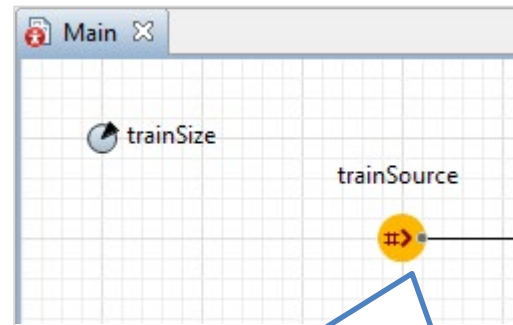
The length of the cars in our model is 14 meters and that is why we need to increase the scale of railway tracks to *1 meter in 1 px* to place the train fully on the track.



Hump Yard. Phase 1. Step 10



Hump Yard. Phase 1. Step 11



❶ New rail car:

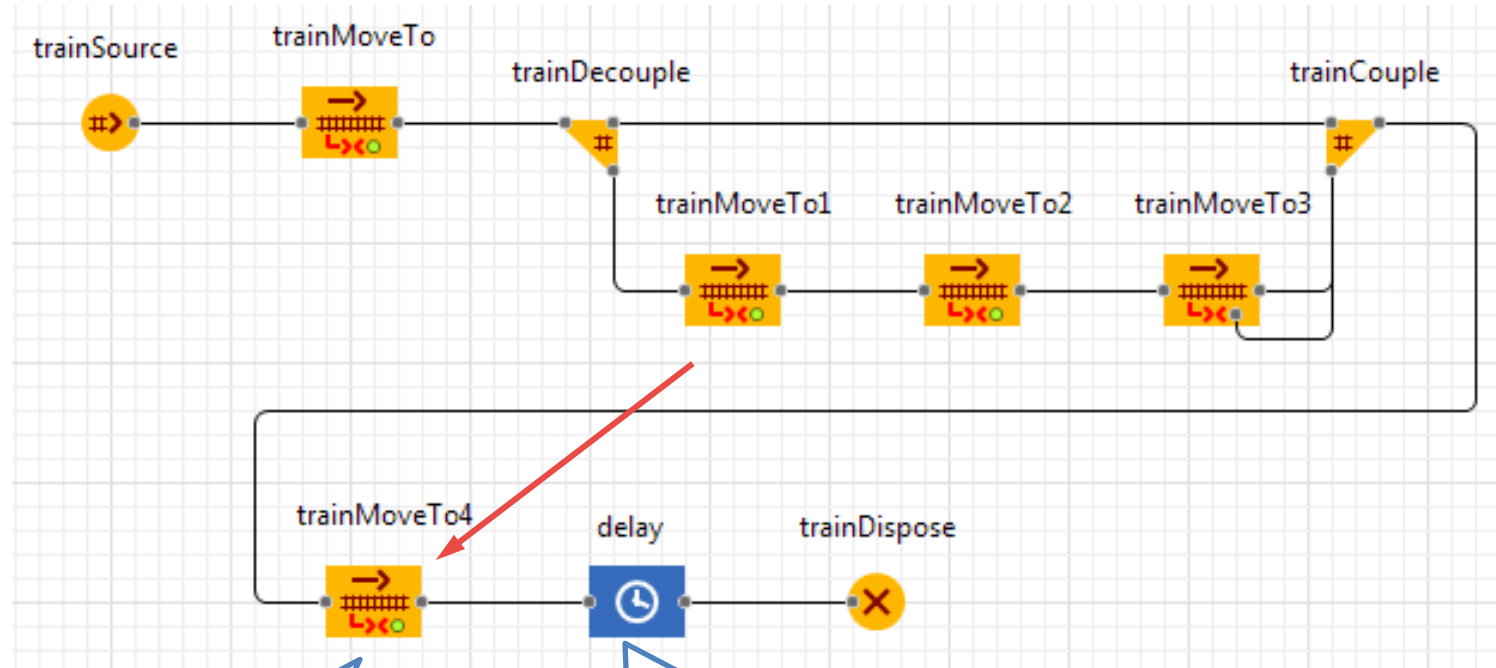
`carindex == 0 ? new Locomotive() : randomlyCreate( OpenCar.class, BoxCar.class, GondolaCar.class, HopperCar.class, TankCar.class )`

- ❶ We want to diversify rail cars. Using the conditional operator, we set *Locomotive* as the type of the first rail car; we call the `randomlyCreate()` function and provide the classes of the rail car types to randomly choose the types for other rail cars in the train.

Conditional operator

- The *conditional* operator allows you to choose between two given alternatives depending on the value of a Boolean expression. In our example, if the “*carindex == 0*” expression evaluates to *true*, then the current rail car is the first car of the train and we set its type to *Locomotive*. Otherwise, the expression evaluates to *false* and we set the rail car type to some other randomly chosen type.

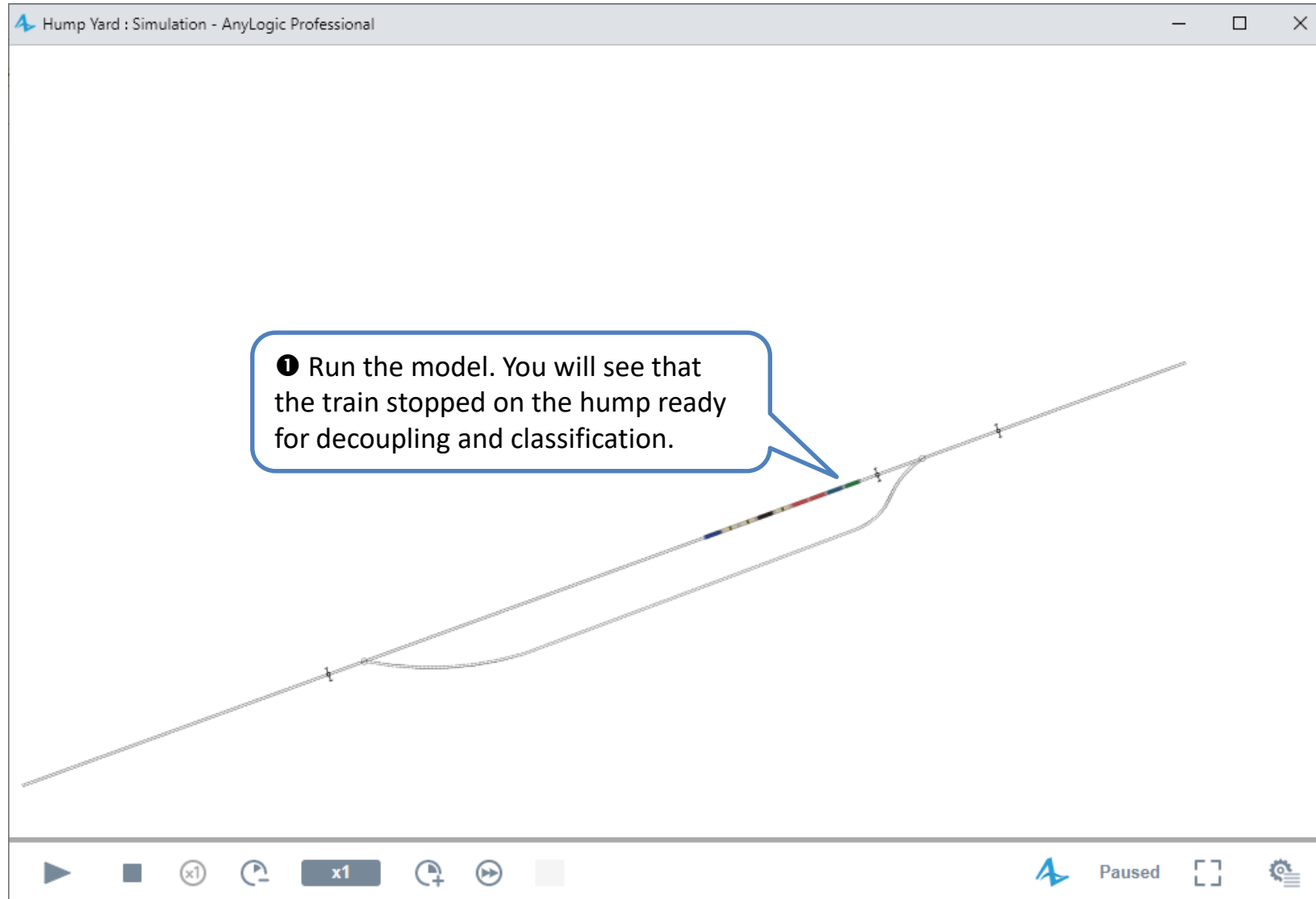
Hump Yard. Phase 1. Step 12



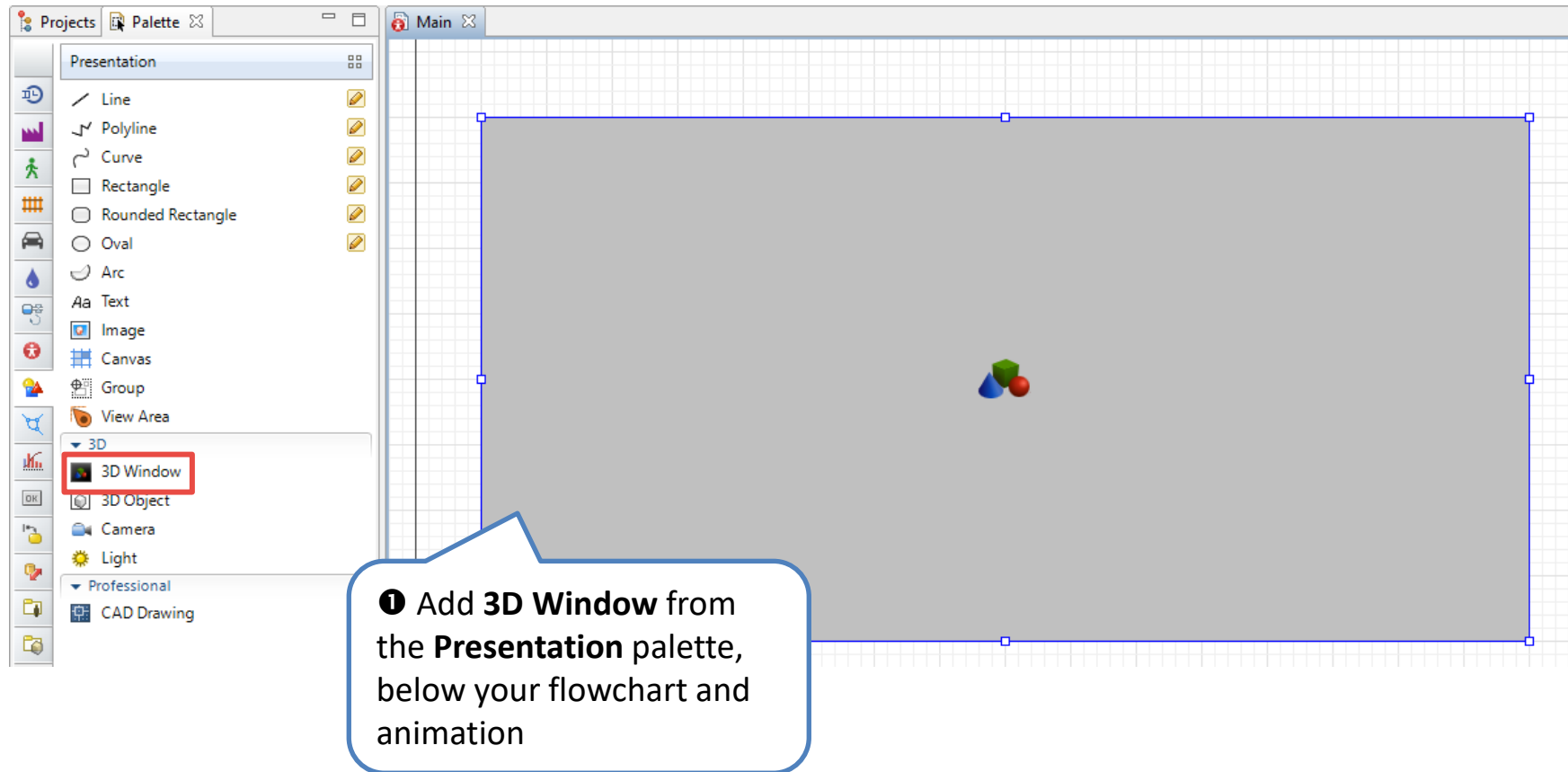
❶ Create one more *TrainMoveTo* by Ctrl+dragging
Check that **Position on track** is *stopLineHump*

❷ *Delay* (Process Modeling Library block)
Delay time: 15 seconds

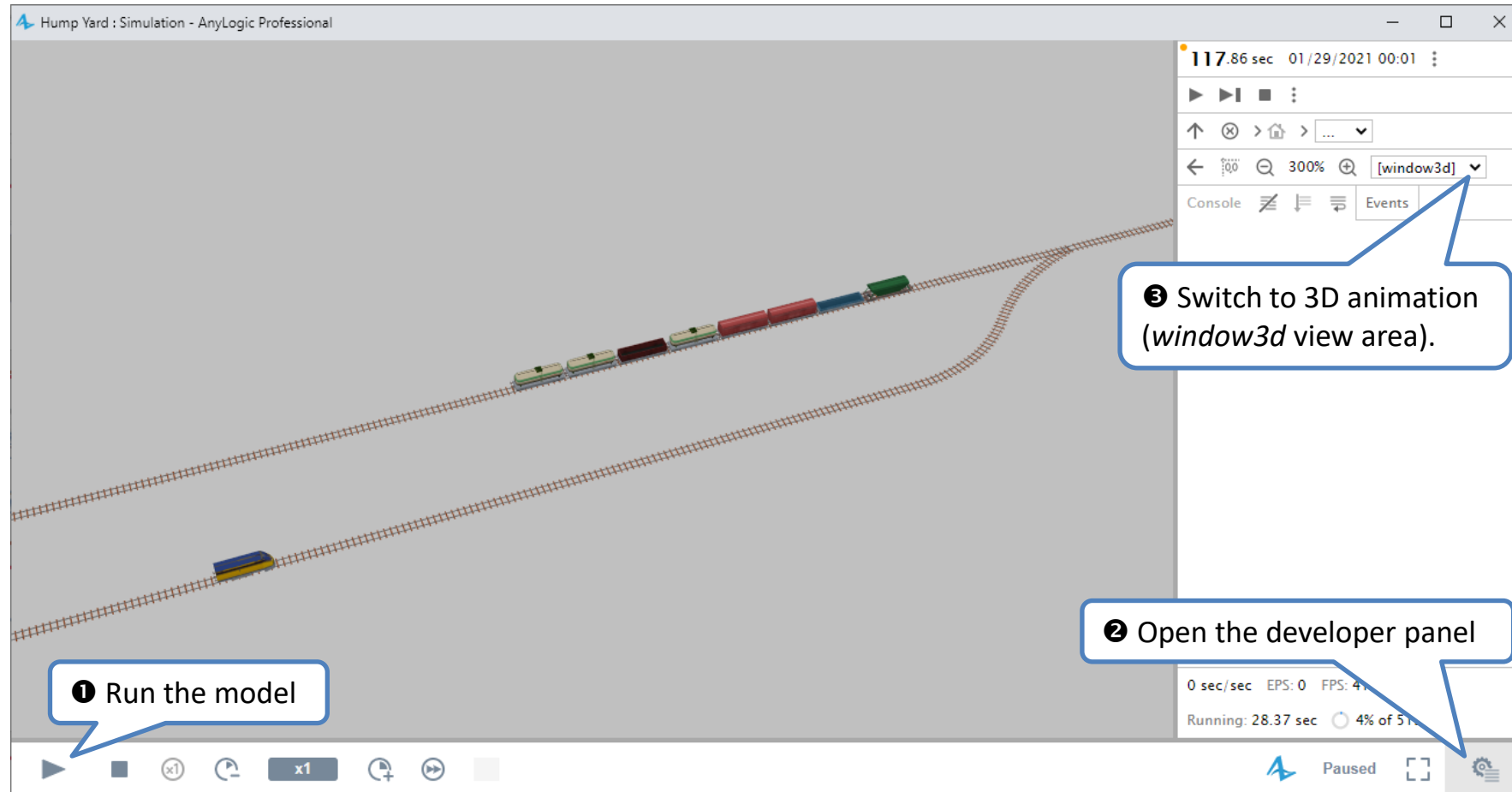
Hump Yard. Phase 1. Step 13



Hump Yard. Phase 1. Step 14



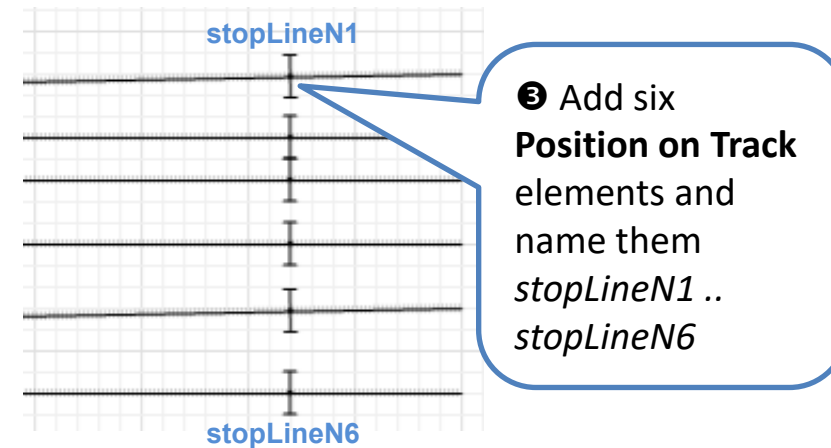
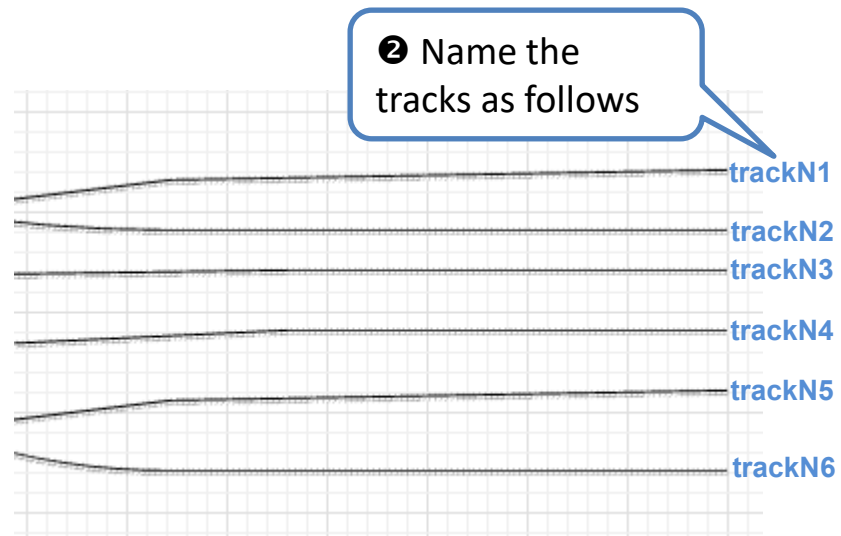
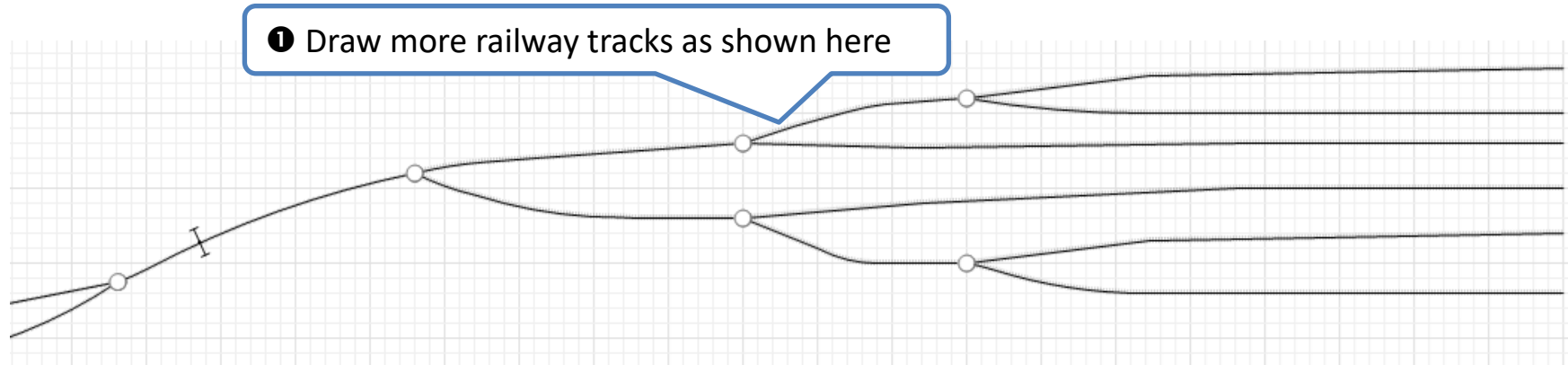
Hump Yard. Phase 1. Step 15



Hump Yard. Phase 2

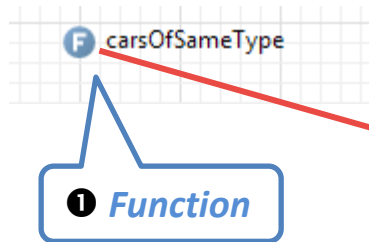
- Now let's simulate how the rail cars are classified and put on different tracks according their types.
- This requires drawing more tracks and adding more blocks in the rail process flowchart.

Hump Yard. Phase 2. Step 1



- ① Draw the destination tracks.
- ② Give meaningful names to the destination tracks.
- ③ Add **Position on Track** on every destination track. That's how we define stop lines for each track. The cars rolling down by gravity onto their destination tracks will stop at these lines.

Hump Yard. Phase 2. Step 2



carsOfSameType - Function

Name: ☒ Show name ☐ Ignore ☒ Visible

☐ Just action (returns nothing)
☒ Returns value

Type:

▼ Arguments

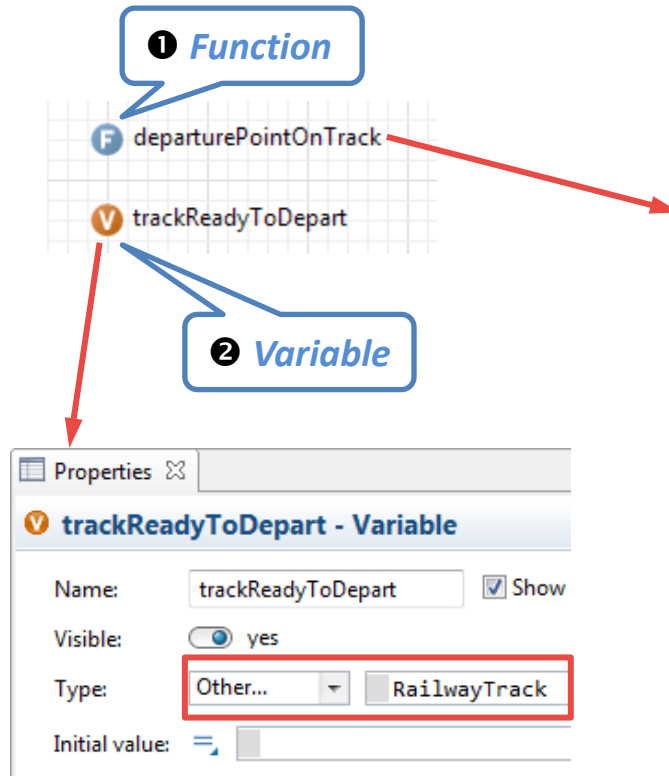
Name	Type
train	Train

▼ Function body

```
Agent firstCar = train.getFirst();
int n = 1;
while( train.getCar(n).getClass() == firstCar.getClass() )
    n++;
return n;
```

- ❶ The train arriving to the hump yard may contain several cars of the same type in sequence. Those cars can be rolled to their track at once. This function calculates the number of successive cars of the same type in the train standing on the hump.

Hump Yard. Phase 2. Step 3



Properties

departurePointOnTrack - Function

Name: ☒ Show name ☐ Ignore

Visible: ☒ yes

☐ Just action (returns nothing)
☒ Returns value

Type:

Arguments

Name	Type
train	Train

Function body

```
Agent car = train.getCar(0);  
if( car instanceof HopperCar ) return stopLineN1;  
if( car instanceof BoxCar ) return stopLineN2;  
if( car instanceof OpenCar ) return stopLineN3;  
if( car instanceof GondolaCar ) return stopLineN4;  
if( car instanceof TankCar ) return stopLineN5;  
return null;
```


- ❶ Let's define a function that will determine the destination track for the rail cars in the classification bowl (the tracks where the cars of this particular type are sorted).

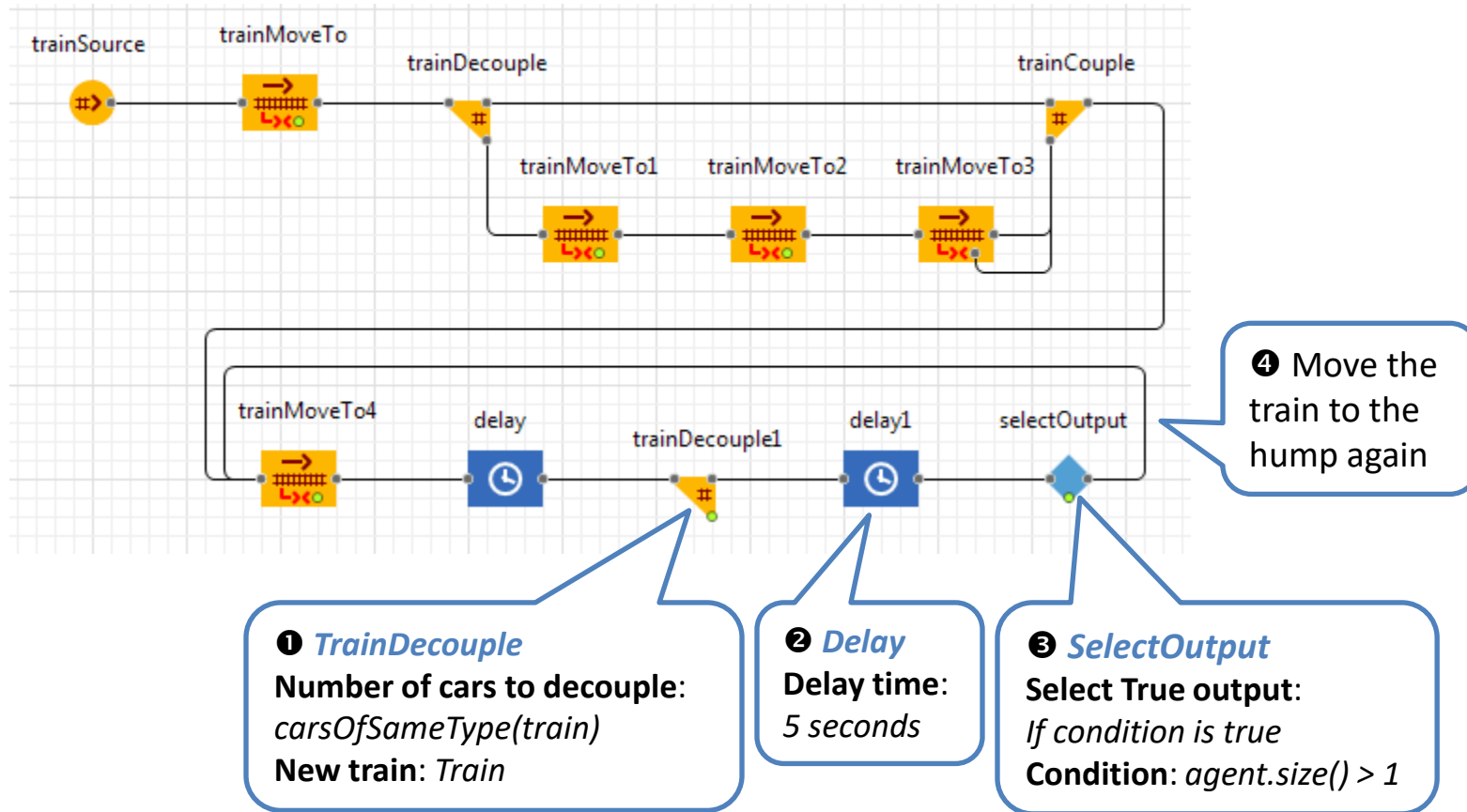
This function takes the train as the argument, then checks the type of the first car of the train (since usually train consists just of a single car).

We find the type of the car with the help of *instanceof* Java operator. It evaluates to *true* if the object is an instance of the specified class.

Finally, the function returns **Position on track** that depends on the car type.

- ❷ This variable stores the reference to the track where the required number of cars of the same type is ready to be taken by a loco from the yard.

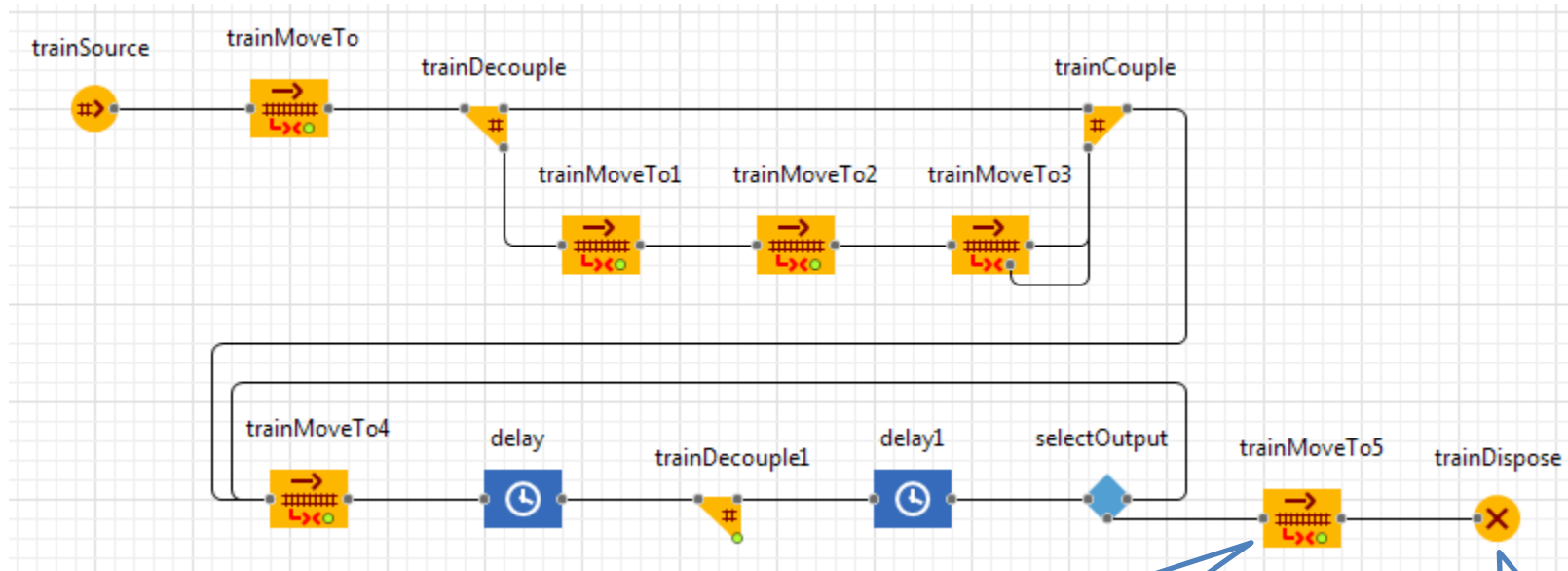
Hump Yard. Phase 2. Step 4



Now let's add new logic to our flowchart.

- ① This **TrainDecouple** block decouples the cars when the train is on the hump. The train may contain several cars of the same type in sequence. Those cars are rolled to their track at once. (We find the number of the cars to be rolled by the function *carsOfSameType* created on the previous step.) The decoupled train consists of single cars, or some coupled cars of the same type in a block. The destination is defined by the function *departurePointOnTrack* we've defined on the previous step.
- ② There is a short delay before the train continues decoupling.
- ③ Now we should determine whether the train contains more freight cars to be decoupled. This is done by **SelectOutput** block. We check whether the train contains more than one car (remember, that one car is the loco itself). If it is so, the train enters the block **TrainDecouple** again. The action repeats until the train contains just one loco ready to depart from the yard.

Hump Yard. Phase 2. Step 5



① *TrainMoveTo*

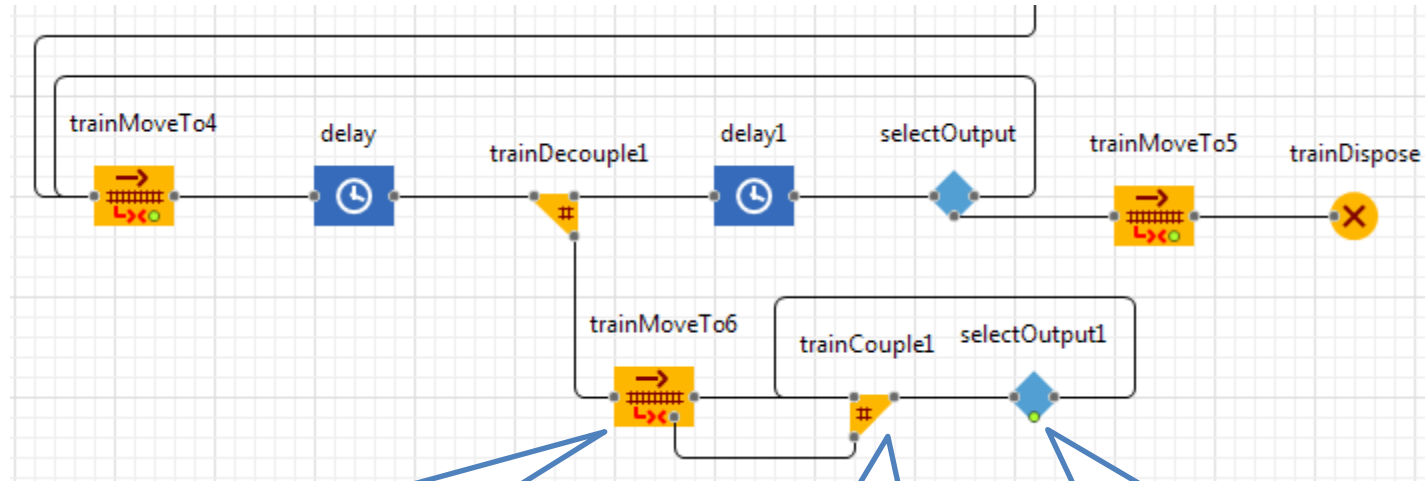
Route is: Calculated automatically...
Target is: A given offset on a track
Railway track: trackN6

② *TrainDispose*

Now let's simulate how the loco leaves the yard.

- ① We use **TrainMoveTo** again to simulate train movement. As a destination point, we specify the name of the lower track (*trackN6*).
- ② Finally, the loco is removed from the model by **TrainDispose** block.

Hump Yard. Phase 2. Step 6



① *TrainMoveTo*

Route is: *Calculated automatically...*

Target is: *A given position on track*

Position on track:

departurePointOnTrack(train)

Finish options: *Decelerate and stop*

② *TrainCouple*

③ *SelectOutput*

Select True output:

If condition is true

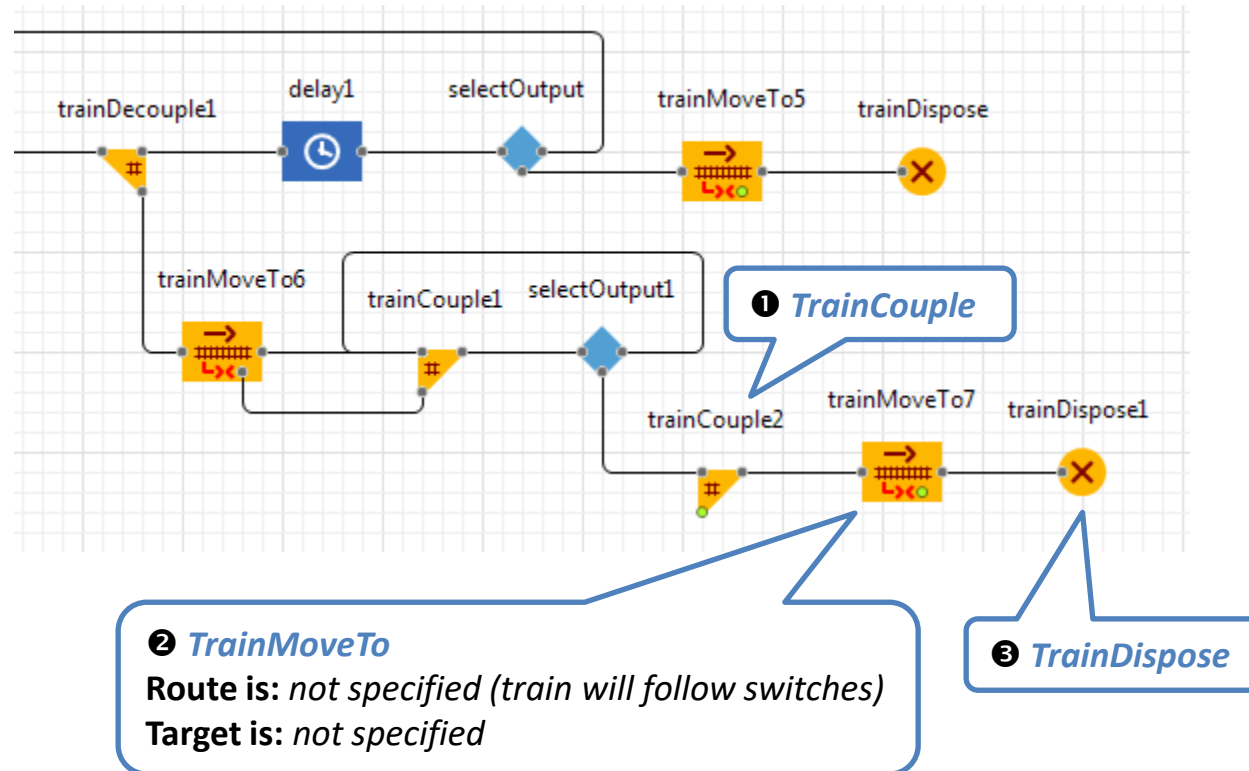
Condition:

agent.size() < trainSize-1

Now we want to simulate how rail cars roll by gravity onto their destination tracks in the classification bowl (the tracks where the cars are sorted).

- ❶ The movement (as always) is simulated with *TrainMoveTo*. The train consists of single cars, or some coupled cars of the same type in a block. The destination is defined by the function *departurePointOnTrack* we've defined earlier.
- ❷ If there are any other cars on the same destination track, they are assembled together with the just rolled cars.
- ❸ The cars continue assembling until the required number of cars is on the track (then a locomotive arrives here and takes them away from the yard). This *SelectOutput* block checks whether the assembly should be stopped or not.

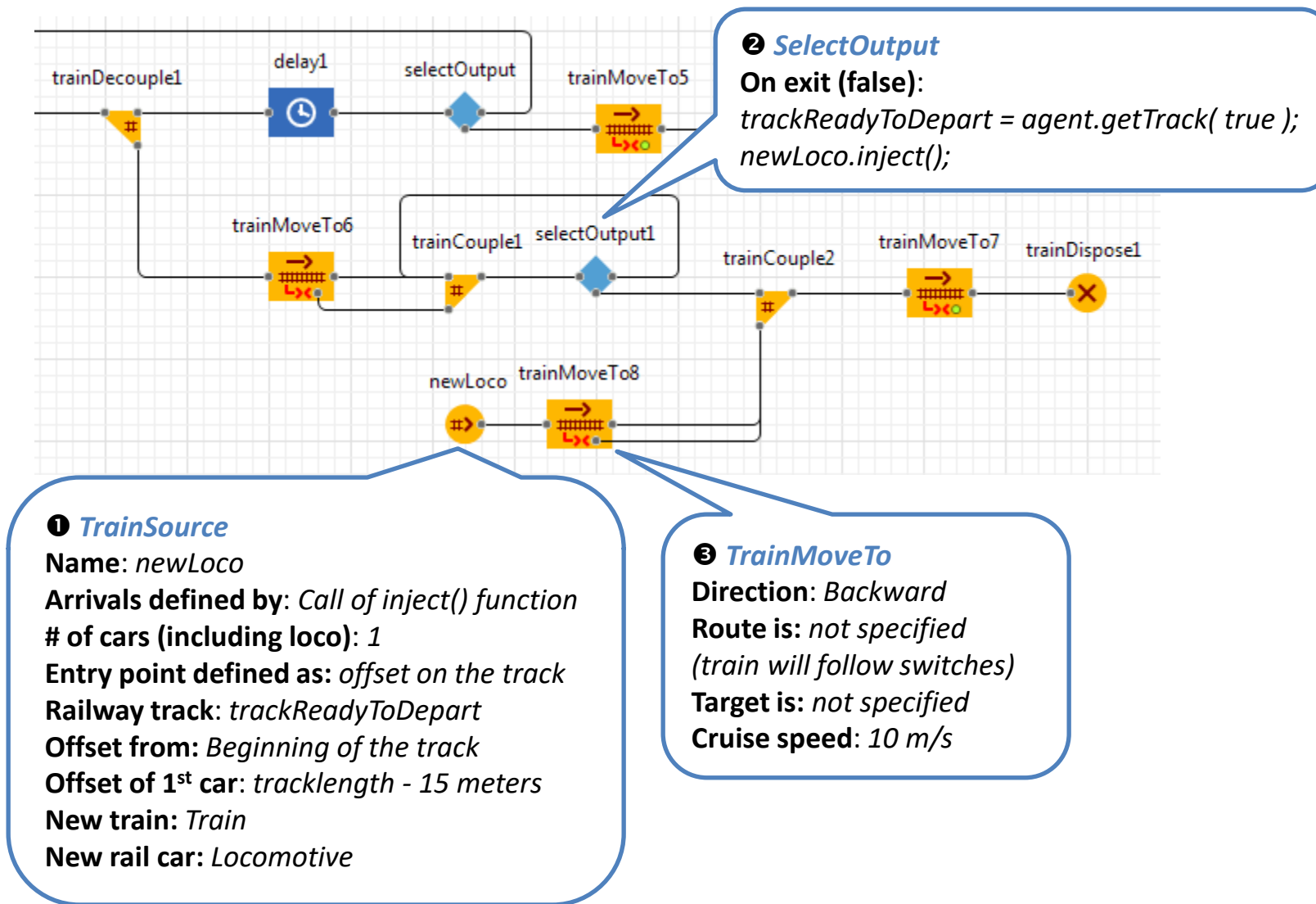
Hump Yard. Phase 2. Step 7



Now let's add a flowchart branch simulating how the ready train combined from the cars of the same type leaves the yard.

- ❶ First, we couple the cars with the loco (the loco logic will be defined on the next step).
- ❷ Then the train leaves the yard (this is modeled with a common *TrainMoveTo* block).
- ❸ Finally we remove the train from the model with the *TrainDispose*.

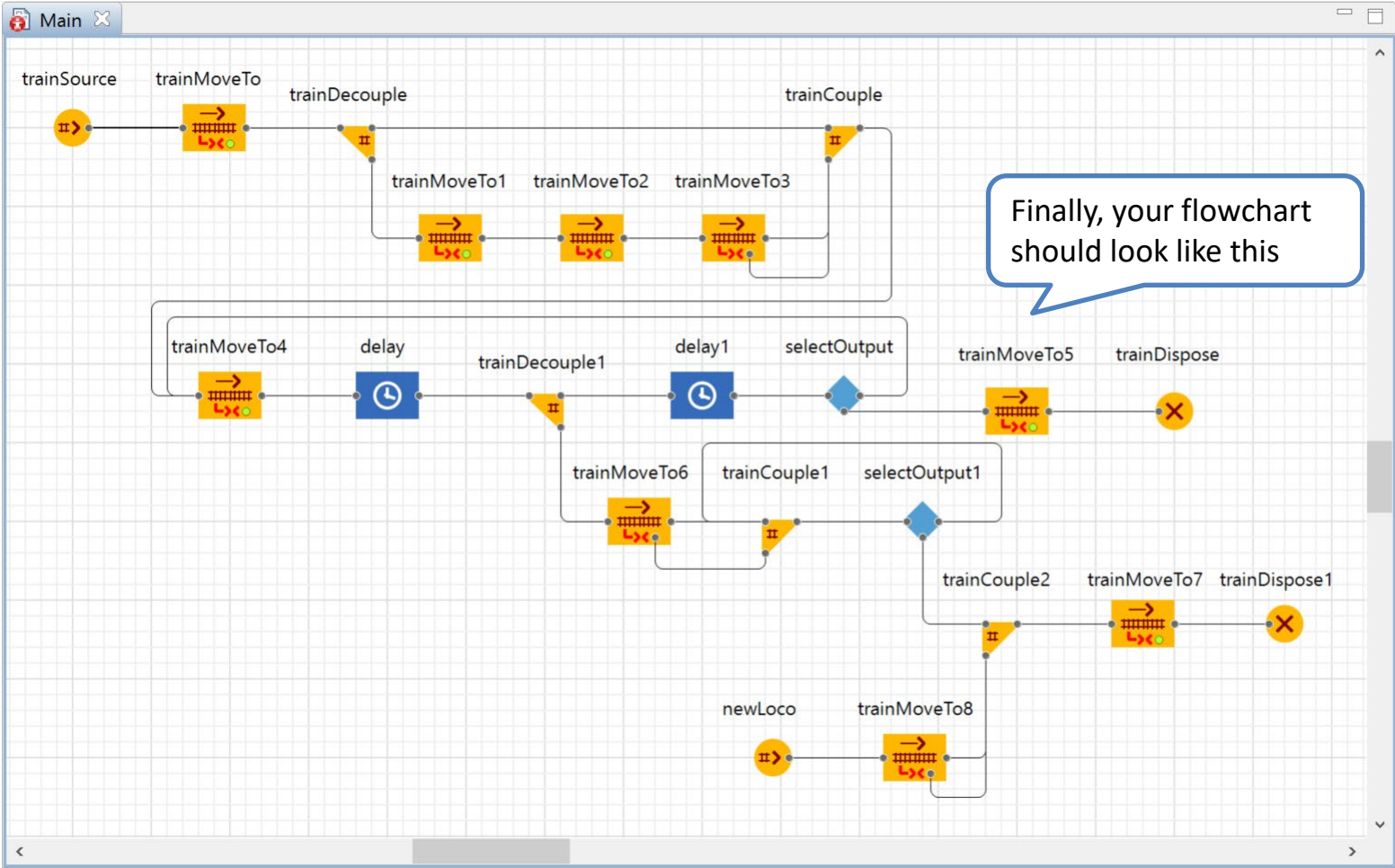
Hump Yard. Phase 2. Step 8



Now we want to simulate how locomotives take ready trains combined from the cars of the same types from the yard.

- ❶ First, let's add **TrainSource** block to model locomotive arrival. The train consists just of one railcar (the loco itself). Choose the manual train generation mode since this block will create trains only when the new train is ready to depart from the yard. The loco should appear on the track defined by the variable *trackReadyToDepart* (we calculate the variable value on the next step).
- ❷ This block checks whether the required number of the cars of the same type is already on the railway track, or not yet. Define the **On exit (false)** action. It is executed when the required number of rail cars is reached.
The line *trackReadyToDepart = agent.getTrack(true);* stores the current track to the variable *trackReadyToDepart*.
The line *newLoco.inject();* generates new locomotive. Being created, the loco follows to the block we create next.
- ❸ This **TrainMoveTo** block simulates how the loco moves to the cars ready to depart. Please note that the movement direction is *Backward* (the loco moves in the direction opposite to the track direction, from right to left). Then the loco couples with the cars and the resulting train passes to the flowchart branch simulating how the train leaves the yard.

Hump Yard. Phase 2. Step 9



Hump Yard. Phase 2. Step 10

